

Can Coding Agents Manage Their Own Memory?

Stanford CS224N Custom Project

Sameer Agrawal
Department of Computer Science
Stanford University
sameer06@stanford.edu

Ryan Wang
Department of Computer Science
Stanford University
wangrj@stanford.edu

Jerry Wang
Department of Computer Science
Stanford University
jwang028@stanford.edu

Abstract

Although agents built with large language models (LLMs) have shown great promise in autonomously solving real-world coding tasks, many challenges remain. Coding agents must be able to maintain a memory of both high-level and low-level details about large codebases, and context management is still an active open question. Current frameworks do not adequately address memory management in situations when context size is limited, with existing approaches defaulting to heuristics like summarizing upon reaching a token threshold. In particular, these approaches use predefined criteria to decide when to update the agent’s memory, instead of letting the agent use reasoning to manage its own memory. We propose a novel framework that leverages the agent’s own reasoning capabilities to make key decisions on when to perform memory operations. Our framework divides a single development cycle into multiple disjoint phases, and equips the agent with a multi-layer memory system in which it has full control over when to make updates. We achieve superior performance compared to limited-context heuristics, with a solve rate on SWE-bench tasks comparable to unlimited-context mini-SWE-agent. This approach demonstrates the potential of self-managed memory for coding agents, especially for long-context tasks.

1 Key Information to include

- Mentor: Arpandeeep Khatua
- Late Days (if you are using any): 2

Team member	Late days remaining
Sameer	6
Ryan	6
Jerry	6
Total pooled	18
Team size	3
Late days applied for the report	2

2 Introduction

In recent years, the rapid development of large language models (LLMs) has brought about a significant increase in the utility of AI agents. These agents work in a feedback loop, gathering information about their environment and reasoning with this context to produce actions [1]. This

need for agent actions to be informed by environmental context naturally necessitates a *memory* system, which the agent uses to keep track of what it has learned and the actions it has taken. For coding agents in particular, memory presents a unique challenge, since the agent must understand and remember information about a potentially vast codebase at various levels of abstraction.

Frameworks such as SWE-agent and mini-SWE-agent [2] solve coding tasks in steps, where the agent runs a bash command on each step and uses the output to inform future steps. The only component of mini-SWE-agent’s memory is its trajectory, a chronologically ordered list of past actions and outputs which could potentially grow endlessly as the model runs.

However, in practice, agents do *not* have unlimited memory. In fact, in many situations, imposing a memory size limit is advantageous in terms of model limitations, performance [3], and out of practical concerns for API costs. Yet, in these limited-context scenarios, modern coding agents still use heuristics to manage memory size, such as summarizing when a context window fills.

We hypothesize that agents can manage their own memory through reasoning, and we propose a novel memory management system for coding agents that leaves key decisions for reducing memory size up to the agent itself. Our framework breaks up a coding task into short *phases*, modeled after segments of a typical software development cycle. The agent is also given multiple layers of memory, representing different contexts, which it updates or resets during phases. Crucially, the *agent* decides itself when to switch phases, and effectively, when to reset components of its memory.

We show that our framework beats widely-used heuristics for limited-context memory management on tasks from the SWE-bench coding benchmark, and achieves comparable performance to unlimited-memory mini-SWE-agent while limiting context growth. This framework demonstrates that self-managed memory could enable coding agents to autonomously handle larger and more complicated tasks in the future.

3 Related Work

LLMs have been known to struggle with extremely large quantities of information since they were first developed. Even as the context windows of LLMs increase to millions of tokens today [4], LLMs still struggle to retain information from their entire context [3].

Numerous fixes have been proposed to help mitigate this problem. For example, Li, et al. suggest compressing the context by allowing an LLM to summarize its own context and prune redundant information [5]. Wu, et al. introduced the idea of giving an LLM external memory storage in the form of (key, value) pairs that it could write to and read from at inference time [6].

Additionally, as tool use in agents has become more widespread [1, 7], this has both allowed and necessitated the development of more complex memory systems. For example, Hu, et al. propose allowing an LLM to store and manipulate its memory in a series of SQL databases [8]. Packer, et al. take inspiration from operating systems, allowing the LLM to write/load memory to/from a “disk”, and change its own context using this retrieved information [9].

Work has also been done on allowing LLMs and agents to develop their own memory-management strategies. For example, Zhang et al. propose a “recursive language model” that can write code in a REPL (Read-Eval-Print Loop) environment to understand and process its own extremely long prompts [10]. Xiong et al. extended this idea to agentic memory, proposing a “meta-agent” that can write its own implementations of certain basic memory-retrieval functions [11]. Both these papers demonstrate improved performance when using the LLM’s strategies over human-designed baselines.

However, there is very little work on endowing coding agents, specifically, with similar memory management capabilities. For example, SWE-agent, one of the state-of-the-art frameworks for agents (based on performance on the SWE-bench dataset [12]) manages long contexts by simply collapsing observations preceding the last five into a single line [2]. Even this is omitted in the simpler mini-SWE-agent, which simply appends observations to the agent’s context window with no limit.

Therefore, while there has been extensive work on LLM and agentic memory management, including having the model devise its own memory-management systems, ways to apply these methods to coding agents have not been nearly as well-explored.

4 Baseline Methods

While the majority of our project is centered around a custom memory-management protocol for mini-SWE-agent described in section 5, we also implemented baselines for our method.

Our first baseline is *recency truncation*, in which we truncate the agent’s previous trajectory whenever the trajectory token count exceeds a predetermined threshold. We also implemented a *summarization* baseline, in which the agent summarizes its entire previous trajectory anytime the trajectory length exceeds the token threshold. However, the agent’s most recent step is kept unchanged to ensure that the agent still has an accurate memory of its latest action.

We ran both of these baselines with token thresholds of 4096 and 8192 tokens.

5 Our Method: Phase Memory

We discuss our novel coding agent framework, which makes use of our proposed *phase* system (5.1) and a multi-layer agent memory design (5.2). Together, these components allow for efficient memory management and reduce the need for long context windows.

5.1 Phases

In our framework, the agent has three types of phases that it cycles through: *exploration*, *execution*, and *validation*. Each phase is short, and highly focused on a very specific objective.

With our phase framework, the agent no longer needs access to its entire previous trajectory, reducing the size of its context window. Instead, it needs only a transcript of the trajectory from the current phase and access to memory structures with important information from previous phases.

5.1.1 Phase Types

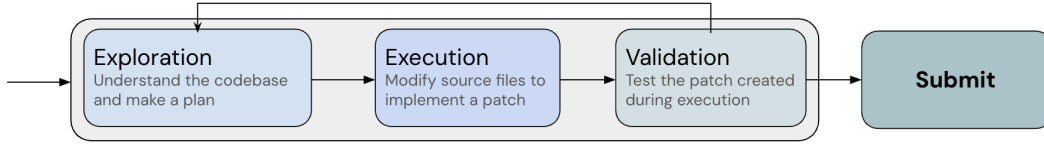


Figure 1: Phases in our agent framework.

The *exploration* phase is the phase the agent always starts in. In this phase, the agent’s only goal is to explore the codebase and prepare a plan for a specific patch attempt. An exploration phase ends when the agent determines that it has gathered sufficient information to create an *exploration handoff* (section 5.2), which includes a concrete plan for the following *execution* and *validation* phases.

The *execution* phase follows an exploration phase. In this phase, the agent’s sole job is to modify source code files to implement a single patch attempt, following the plan from the exploration handoff. Once the agent completes a patch, it should immediately pass control to the *validation* phase.

In the *validation* phase, the agent’s only responsibility is to thoroughly validate the patch produced by the preceding execution phase, using the plan provided in the exploration handoff. At that point, if all validation tests have succeeded, the agent proceeds to patch submission, while if any tests fail, the agent returns to exploration to diagnose the issue and restart the process of producing another patch.

5.1.2 Phase Switching Tool

We implement phase switching by giving the agent an additional custom `switch_phase` tool. On each step, it calls either `switch_phase` if it has already achieved the goals for the current phase, or `bash` if it decides to continue taking current phase actions. Calling `switch_phase` updates its internal state, including various memory structures (5.2). The decisions of when to switch phases are completely left to the agent itself. More details are given in 5.2 and 5.3.

5.2 Multi-Layer Memory

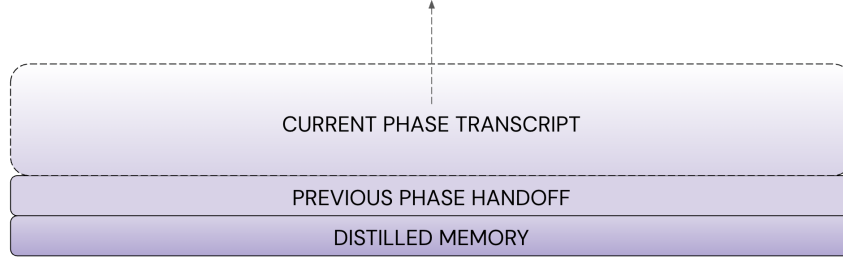


Figure 2: Agent memory during a phase. Phase transcript length grows with phase duration.

In our framework, the agent manages and dynamically updates a multi-layer memory system. Its memory consists of an objective laid out by the previous phase in a *handoff*, a global memory storage called the *distilled memory*, and a phase transcript with what it has accomplished in the current phase. This distinction between distilled memory and phase handoffs and transcripts is similar to the distinction between “cache” and “disk” memory for agents described in Packer, et. al [9].

Both the phase handoff and distilled memory are in a natural-language format, and they are generated by the agent framework using LLM calls. At any point, the agent has access to all three memory structures through its prompt, enabling it to make decisions that are informed by both recent and long-term context.

5.2.1 Phase Handoff

The purpose of a *phase handoff* is to provide an immediate, focused objective for the next phase based on what was learned or attempted in the current phase. A phase handoff is created at the end of a phase, after `switch_phase` is called and before the completion of the phase switch. There are two types of phase handoffs.

An *exploration handoff* is created by an exploration phase right before it switches to execution. It has two sections, an `EXECUTION PLAN` and a `VALIDATION PLAN`, each of which contains a numbered sequence of steps to be carried out by the following execution and validation phases.

An *execution report* is created by a failed validation phase switching back to exploration. This report contains the exact patch that failed validation, as well as the specific validation commands that failed for that patch. The aim of the execution report is to help the next exploration phase create a new patch plan that avoids the mistakes of the failed patch.

5.2.2 Distilled Memory

```
# CONFIRMED RELEVANT FILES / CODE LOCATIONS
- 'astropy/io/ascii/rst.py': Contains the 'RST' class which is missing 'header_rows' in its '__init__' method and has a 'write' method that assumes a fixed index for the table border line.
- 'astropy/io/ascii/fixedwidth.py': The parent class 'FixedWidth' is defined here. It correctly handles 'header_rows' in its '__init__'.
- 'astropy/io/ascii/core.py': Contains the '_get_writer' function which instantiates the writer class with '**kwargs', leading to the 'TypeError'.
```

Figure 3: Distilled memory showing CONFIRMED RELEVANT FILES / CODE LOCATIONS

The *distilled memory* stores persistent information across phases. While old handoffs are discarded and trajectory messages from previous phases are not accessible to the agent, the information in the distilled memory is kept through phase switches. As such, the distilled memory is the only resource the agent has to safely store information it may need multiple phases later. Like phase handoffs, the distilled memory is updated at the end of exploration and validation phases.

We opted for a minimalist design in the distilled memory, as its main purpose is to prevent the agent from repeating actions it has taken in the past. To this end, our distilled memory is specified only by four section headers: `CONFIRMED RELEVANT FILES / CODE LOCATIONS`, `CONFIRMED BUG MECHANISM / WORKING HYPOTHESES`, `REPRODUCTION AND VALIDATION COMMANDS`, and `FAILED PATCH ATTEMPTS`.

5.2.3 In-Phase Context

In addition to a phase handoff and the distilled memory, which contain information from prior phases, the agent must also have knowledge of what it has accomplished in the current phase. To keep our design simple, our in-phase context is a transcript of the current phase messages, lightly formatted to exclude irrelevant metadata.

5.3 Control Flow

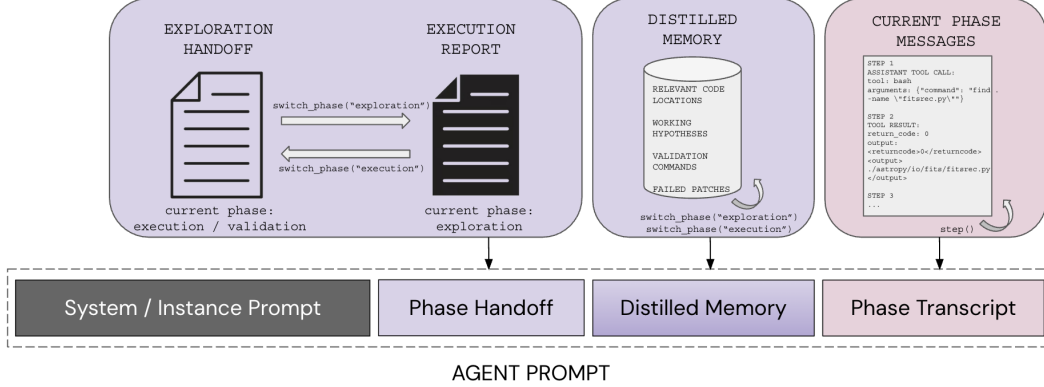


Figure 4: Complete agent prompt and memory control flow.

Figure 4 displays all components of agent memory in our framework and control flow for memory updates. On each step, the complete agent prompt is built from the system/instance prompt, the phase handoff from the previous phase, the distilled memory, and the current phase transcript.

Only a single phase handoff is kept in the agent’s memory at all times. During an execution or validation phase, this is the previous exploration handoff, and during an exploration phase, this is the previous execution report, if it exists. Upon switching from exploration to execution or validation to exploration, the old handoff is discarded and replaced by a newly created handoff.

Similarly, distilled memory is updated before switching to a new phase, adding the information learned during the current phase and deleting invalidated hypotheses. Finally, the phase transcript is appended to on every single step and reset at the start of every single phase, including validation.

6 Experiments

6.1 Data

Our dataset consists of 51 GitHub issues from the SWE-bench Lite benchmark, which is a set of 300 real-world software issues collected from several popular Python GitHub repositories [12].

To select tasks for our dataset that represented a wide range of difficulty levels, we ran mini-SWE-agent on the first 100 samples of SWE-bench Lite. We then divided these 100 samples into three categories depending on how many steps mini-SWE-agent took to finish: “easy” (1 - 23 steps), “medium” (24 - 46 steps), and “hard” (47 - 70 steps), and randomly sampled 17 datapoints from each of these three categories. While step count does not directly indicate difficulty, we note that it is strongly correlated with solve rate, as shown in Table 1.

Category	Avg. Step Count	Solve Rate
Easy	19 steps	11/17 (65%)
Medium	34 steps	8/17 (47%)
Hard	60 steps	2/17 (12%)

Table 1: Dataset breakdown. Step count and solve rate were measured with default mini-SWE-agent.

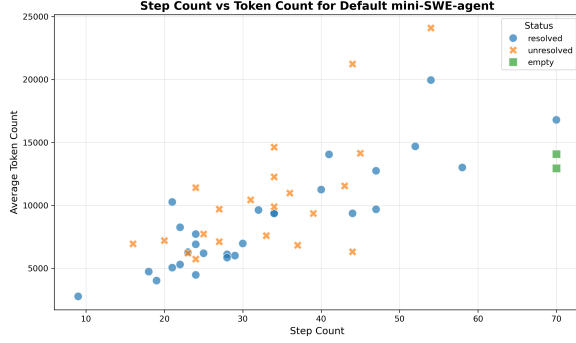


Figure 5: Step count vs. avg. token count for default agent.

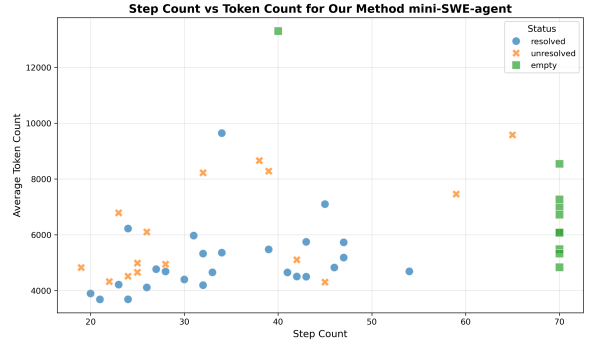


Figure 6: Step count vs. avg. token count for our method.

6.2 Evaluation method

All of our experiments were run on mini-SWE-agent version 2 using Gemini 3.0 Flash as our base LLM. We constrained all runs to a maximum of 70 steps. The main metrics for our methods were solve accuracy (number of correct patches) and average input token length across all runs.

6.3 Results

Method	Correct	Incorrect	Incomplete	Accuracy
Default	28/51 (55%)	21/51 (41%)	2/51 (4%)	28/49 (57%)
4K Truncation	10/51 (20%)	6/51 (12%)	35/51 (69%)	10/16 (63%)
8K Truncation	22/51 (43%)	14/51 (27%)	15/51 (29%)	22/36 (61%)
4K Summary	14/51 (27%)	10/51 (20%)	27/51 (53%)	14/24 (58%)
8K Summary	23/51 (45%)	19/51 (37%)	9/51 (18%)	23/42 (55%)
Phase Memory (Ours)	25/51 (49%)	15/51 (29%)	11/51 (22%)	25/40 (63%)

Table 2: Accuracy scores and token counts for various memory management techniques.

Our main results are in Table 2. Phase Memory achieves a correct solve rate of 25/51 (49%), which is better than both the truncation and summarization baselines for 4K and 8K tokens. This solve rate is close to that of default mini-SWE-agent with unlimited memory, which achieves 28/51 (55%). Yet, Phase Memory achieves its performance with a far smaller number of tokens, with an average token count of 5812 compared to 9708 tokens for default mini-SWE-agent.

Notably, Phase Memory achieves the highest accuracy of submitted patches (correct/total complete) out of all methods, at 25/40 (63%). We believe this high accuracy can be attributed to the dedicated validation phase, which may block incorrect patches from being submitted.

In addition, Figure 6 shows that Phase Memory is highly scalable, with average token count remaining relatively constant as step count increases. Moreover, the majority of tasks complete with an average token count of less than 8K, showing that Phase Memory is well suited for limited-context situations. This is in stark contrast with Figure 5, which shows average token count increasing sharply for default mini-SWE-agent as step count increases.

7 Analysis

An early challenge was enforcing phase boundaries—ensuring that the agent only performed actions specific to its phase. The agent would often write to source files during exploration, when it should only be reading and planning. However, with strict prompting, we were mostly able to mitigate this. In our final runs, the agent adhered better to phase-specific actions. Below is an example of a condensed transcript of an exploration phase.

```

STEP 1: find . -name "fitsrec.py" # search for the fitsrec.py file
STEP 3: grep -nC 5 "'D' in format" astropy/io/fits/fitsrec.py # search fitsrec.py for "'D' in format"; show 5 lines around each match
STEP 5: sed -n '1240,1270p' astropy/io/fits/fitsrec.py # print lines 1240-1270 of fitsrec.py
STEP 7: grep -n "def _scale_back" astropy/io/fits/fitsrec.py # find the _scale_back function in fitsrec.py and print its line number
STEP 9: sed -n '1196,1266p' astropy/io/fits/fitsrec.py # print lines 1196-1266 to view the full _scale_back implementation
STEP 11:
  THOUGHT:
  ...
  I will now create a reproduction script to confirm the bug and verify that the 'E' is not being replaced by 'D' for 'D' format ASCII tables.
  After that, I will have enough information to switch to the execution phase.
  ...
  cat <<EOF > reproduce_bug.py ... EOF # create a new Python file reproduce_bug.py
  python reproduce_bug.py # run the reproduction script to trigger and observe the bug
STEP 13:
  TOOL RESULT: ModuleNotFoundError: No module named 'numpy'
STEPS 14-21: # find the commands to import numpy and run the reproduction script
MODEL RESPONSE AFTER STEP 21:
  THOUGHT:
  ...
  I have confirmed the bug with a reproduction script. The 'E' exponent is not being replaced by 'D' in ASCII tables with 'D' format
  because the 'replace' call is not in-place.
  ...
  switch_phase("execution") # call the switch_phase tool to switch to execution

```

As can be seen in the transcript, most tool calls involved exploration and file-reading commands like `find`, `grep`, and `sed`, consistent with our instructions for exploration. However, starting at step 11, the agent created a reproduction script, which is not strictly an exploration action. More generally for all tasks, most agent actions were directly relevant for the current phase, but some actions would fit better in another phase. While not all such cases were ultimately harmful for task completion, this is undesirable because longer phases leads to longer phase transcripts, and hence, memory size.

Despite occasional violations of phase instructions, phases generally remained short and focused. Table 3 shows average phase lengths for the SWE-bench run shown in Table 2. Execution and validation are shorter than exploration, since they mostly adhere to the exploration handoff instructions. Overall, the low average phase length of 8.6 steps is beneficial for limited-context settings, since the phase transcript, the largest component of memory (see Table 4), resets on average every 8.6 steps.

	Average Length (Steps)
Exploration	12.3
Execution	6.3
Validation	7.1
Overall	8.6

Table 3: Average phase length in steps by phase type.

Our long-term memory structures were also effective in storing information needed for future phases. We found that our minimalist design, relying largely on LLM reasoning capabilities to fill in broad headers, was able to capture important details in a relatively consistent format. For example, here is part of the exploration handoff on the same task as for the previously shown transcript.

```

# EXECUTION PLAN

1. Bug Hypothesis: The code in 'astropy/io/fits/fitsrec.py' attempts to replace the exponent separator 'E' with 'D' for FITS ASCII tables
   using the 'D' format. However, it calls 'output_field.replace(...)' without assigning the result back. Since 'chararray.replace' (or string replace)
   returns a copy rather than modifying the array in-place, the replacement is lost, and the output incorrectly retains the 'E' separator.
2. Primary Target File: 'astropy/io/fits/fitsrec.py'
3. Target Function: '_scale_back_ascii' (specifically around line 1263-1264).
4. Logic Change:
   - Locate the following block at the end of '_scale_back_ascii':
     # Replace exponent separator in floating point numbers
     if 'D' in format:
         output_field.replace(encode_ascii('E'), encode_ascii('D'))
     ""
   - Modify the call to perform an in-place assignment to the 'output_field' slice:
     # Replace exponent separator in floating point numbers
     if 'D' in format:
         output_field[:] = output_field.replace(encode_ascii('E'), encode_ascii('D'))

```

This execution plan contains all necessary elements: a bug hypothesis, a concrete path to a code file, and the required logic changes, setting the agent up for a straightforward execution phase. In most cases, phase handoffs and the distilled memory were written in formats similar to above. The biggest issue that frequently arisen was hallucinated file paths or validation commands, which could be mitigated by instructing the agent to verify its file paths and validation commands.

Strong evidence of the usefulness of distilled memory is shown in Figure 7. This graph shows the average length of phases in each cycle for the same SWE-bench run as Table 2 and Table 3. The general downward trend in the average length of exploration phases for later cycles shows that the information stored in the distilled memory from previous exploration phases is helping inform later exploration phases. In contrast, the distribution of execution and validation phase lengths is flatter as the lengths of patch creation and validation do not depend on prior knowledge.

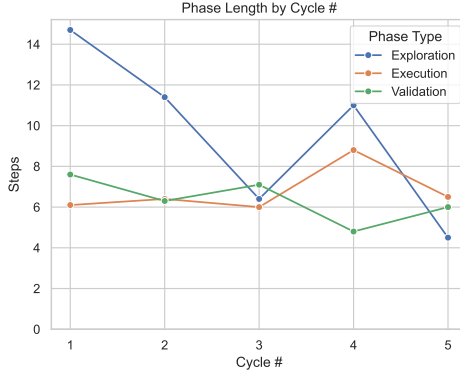


Figure 7: Average phase length by cycle and phase type.

Finally, Table 4 shows the average size of each memory component in tokens and its average share of the prompt. Interestingly, our system/instance prompts made up nearly 40% of the token length of agent prompts, comparable to the phase transcript. Although the average phase transcript length was nearly double (3950 vs. 2123 tokens), the system/instance prompt dominated in the early steps of a new phase, when the phase transcript is short. This was largely due to the specific instructions for each phase and explanations of custom memory structures. We believe that with more fine-tuning, this prompt could be shortened, further improving memory usage.

	Average Length (Tokens)	Average % of Prompt Length
System / Instance Prompt	2123	38.47%
Phase Handoff	405	7.33%
Distilled Memory	472	8.59%
Phase Transcript	3950	45.62%

Table 4: Average length and percentage of final prompt for each prompt component.

8 Conclusion

We introduced a novel memory management system for coding agents that focuses on letting agents handle their own memory. By leveraging a phase-based structure (exploration, execution, validation) and a multi-layered memory design, our framework allows agents to more efficiently handle memory. The results from the SWE-bench coding benchmark show that this approach is comparable to the default agent with unlimited memory, while maintaining a more practical memory size.

However, there are still many areas for further improvement. As mentioned before, stricter phase boundary enforcement, either through stronger prompting or setting hard limits on allowed tool calls, is needed to keep the model focused. The agent also occasionally hallucinated files or test outputs, which may be unavoidable due to the nature of LLMs. However, this could also potentially be mitigated through better prompting and external validations of agent outputs in intermediate steps.

Overall, our work demonstrates that coding agents have the potential to effectively manage their own memory, furthering efforts to build more scalable and efficient AI systems.

Team contributions (Required for multi-person team)

Ryan: Introduction, Our Method, Analysis, implemented base Phase Memory framework

Jerry: Introduction, Ran Experiments, Analysis, Conclusion

Sameer: Introduction, Related Work, Experiments, Results, Implemented Baselines, Dataset

References

- [1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023.
- [2] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024.
- [3] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023.
- [4] Google Gemini Team. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, 2025.
- [5] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353, Singapore, December 2023. Association for Computational Linguistics.
- [6] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers, 2022.
- [7] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.
- [8] Chenxu Hu, Jie Fu, Chenzhuang Du, Simian Luo, Junbo Zhao, and Hang Zhao. Chatdb: Augmenting llms with databases as their symbolic memory, 2023.
- [9] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2024.
- [10] Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models, 2026.
- [11] Yiming Xiong, Shengran Hu, and Jeff Clune. Learning to continually learn via meta-learning agentic memory designs, 2026.
- [12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024.